

ARCHITECTURE AND DEVELOPMENT OF AN ENTERPRISE-GRADE AI CHATBOT WITH SEMANTIC KNOWLEDGE RETRIEVAL

(SYNOPSIS (ES 451))

submitted in partial fulfillment of the requirement
for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

Submitted By:

UJJWAL KUMAR

(Enrollment No: 00818007223)

Under the supervision of:

Mr. UPENDRA PANDEY

(ASSISTANT PROFESSOR)



Affiliated to GGSIP University, New Delhi
Approved by AICTE & Council of Architecture

**Department of Computer Science & Engineering
Delhi Technical Campus**

28/1, Knowledge Park-III, Greater Noida - 201306 (U.P.)

APRIL 2026

TABLE OF CONTENTS

	Page No (in numeric)
Chapter 1: Introduction	3
Chapter 2: Problem Definition	4
Chapter 3: Feasibility Study	5
Chapter 4: Need and Significance	6
Chapter 5: Objectives	7
Chapter 6: Methodology	8
Chapter 7: Hardware and Software Requirements	9
Chapter 8: System Architecture and Design	10
Chapter 9: Project Timeline	11
Chapter 10: Implementation Results	12
Chapter 11: Conclusion and Future Scope	13
Chapter 12: References	14

1. Introduction

In the current era of rapid digital transformation, organizations are generating vast amounts of unstructured data daily. This data, often stored in massive PDF repositories, technical manuals, and policy documents, constitutes what is known as "Dark Data"—information that is collected and stored but seldom utilized because it is difficult to navigate. Traditional search engines used within corporate intranets rely on keyword matching, which often returns irrelevant results or fails to find specific details hidden within a 100-page document. As a final-year student looking at this problem, it became clear that a more "intelligent" way to talk to our data was needed.

The "Enterprise AI Chatbot" project was born out of the necessity to bridge the gap between static document storage and active knowledge retrieval. By utilizing a technique called Retrieval-Augmented Generation (RAG), we can now create a system that doesn't just "search" for a document but actually "reads" and "understands" it to provide a concise, human-like answer. Unlike general-purpose AI like ChatGPT, which might hallucinate or leak sensitive corporate data to the cloud, our system is designed to work locally. This ensures that every byte of information remains within the organization's firewall, providing both intelligence and absolute data sovereignty.

Furthermore, the introduction of Large Language Models (LLMs) like Llama 3.2 has made it possible to run sophisticated reasoning engines on standard workstation hardware. This project explores the integration of these models with vector databases to create a "Private Corporate Brain." The goal is to move away from the "Ctrl+F" era and move toward a conversational era where an employee can simply ask, "What is the reimbursement policy for travel in the 2024 handbook?" and get an immediate, accurate response with citations.

2. Problem Definition

The primary problem addressed by this project is the inefficiency of information discovery in professional environments. When an employee needs specific information, they usually spend 15 to 30 minutes manually scanning through PDFs. This leads to a massive loss in productivity. Moreover, traditional search algorithms do not understand context. For example, if a user searches for "Apple," a traditional system might show documents about the fruit when the user actually meant the technology company. This lack of semantic understanding is a major bottleneck.

Another critical dimension of the problem is data privacy. Most high-performing AI models today are hosted by third parties. For an enterprise, uploading proprietary legal contracts or secret project blueprints to an external server is a non-starter due to compliance laws like GDPR or internal security policies. There is a desperate need for a solution that provides the "power of AI" without the "risk of the cloud." Lastly, the problem of "Hallucination" in AI is a major concern for businesses. Standalone LLMs often provide confident but incorrect answers because they are relying on their training data rather than specific, factual documents. In a corporate setting, an incorrect answer regarding a safety protocol or a legal clause could have disastrous consequences. Therefore, defining a system that "grounds" the AI in actual documents—where it is forced to only answer based on the provided text—is the core technical challenge we are solving here.

3. Feasibility Study

The feasibility of this project was analyzed across three major pillars: Technical, Economic, and Operational. From a technical standpoint, the recent release of quantized model weights (like the 3B and 8B versions of Llama 3) has made local inference highly feasible. Previously, running an LLM required industrial-grade server farms; now, an NVIDIA RTX 3060 or even a high-end CPU with 16GB of RAM can handle these tasks. The availability of open-source frameworks like FastAPI for the backend and ChromaDB for vector storage further solidifies the technical foundation.

Economically, the project is highly viable as it leverages the "Open Source" movement. By avoiding proprietary APIs like OpenAI's GPT-4, the organization saves thousands of dollars in recurring token costs. The only cost involved is the initial hardware setup, which most modern offices already possess in their workstations. There are no hidden subscription fees, and the system can be scaled horizontally by adding more local nodes as the document library grows.

Operationally, the system is designed with the end-user in mind. We have implemented a "One-Click Startup" logic using a .bat file that initializes the Ollama service, the FastAPI backend, and the frontend web server simultaneously. This ensures that the system doesn't require a resident data scientist to maintain it. For a college project, this level of operational simplicity is crucial because it demonstrates that the technology is "ready for the real world" and not just a theoretical prototype.

4. Need and Significance

The need for this project stems from the "Information Overload" experienced by modern professionals. In a typical B.Tech curriculum, we learn about databases and search, but we rarely see how to handle the "unstructured" side of data. This project is significant because it applies cutting-edge NLP research (RAG) to a very practical, boring, but expensive problem: finding a needle in a haystack of PDFs. The significance of using Llama 3.2 locally cannot be overstated. In an era where data is the new oil, keeping that oil within your own tanks is a strategic advantage. This project empowers small to medium-sized enterprises (SMEs) who cannot afford expensive enterprise AI licenses but still need to compete in terms of efficiency. It levels the playing field by providing "state-of-the-art" intelligence on "standard" budgets.

Furthermore, this project serves as a template for "Responsible AI." By implementing source-tracing (where the chatbot tells you exactly which page of which document it found the answer on), we are promoting transparency. This "Trust but Verify" model is what businesses actually want. It changes the chatbot from a "toy" into a "tool." As a student, working on this has provided deep insights into vector mathematics, API orchestration, and the challenges of real-time text processing, making it a significant milestone in my engineering journey.

5. Objectives

The primary objective of this project is to develop a fully functional, end-to-end RAG-based AI system. To break this down into specific, measurable goals:

1. **Architecture Design:** To design a modular architecture that separates document processing, vector indexing, and response generation. This allows for easy updates—for example, replacing the LLM without rebuilding the entire database.
2. **Semantic Search Implementation:** To move beyond keyword matching by implementing Cosine Similarity search using 768-dimensional embeddings. The objective is to achieve a retrieval accuracy of over 90% for relevant document chunks.
3. **Local Inference Engine:** To successfully host and serve Llama 3.2 via Ollama, ensuring that response latency remains under 5 seconds for a typical query on consumer hardware.
4. **UI/UX Development:** To create a modern, professional web interface using "Glassmorphism" design principles. The goal is to make the tool as easy to use as a Google search bar, with clear indicators for document upload status and AI "thinking" states.
5. **Robust Error Handling:** To implement a "Dual-Mode" system where the chatbot can intelligently inform the user if the answer is NOT in the documents, rather than making things up. This objective is key to maintaining the system's reliability in an enterprise setting.

6. Methodology

The methodology follows a "Pipeline-First" approach, which is divided into five distinct stages.

Stage 1: Document Pre-processing. We use the pypdf library to extract raw text from PDF files. This is harder than it sounds because PDFs often contain complex layouts, tables, and multi-column text. Our methodology includes a "Cleaning" step to remove artifacts like page numbers and headers that could confuse the AI.

Stage 2: Semantic Chunking. Large documents cannot be fed into an AI all at once. We use a RecursiveCharacterTextSplitter. The methodology specifies a chunk size of 1000 characters with a 200-character overlap. This overlap ensures that if a sentence is cut in half, the context is preserved in the next chunk.

Stage 3: Vector Embedding. Each text chunk is passed through the nomic-embed-text model. This turns words into a list of numbers (vectors). These vectors represent the "meaning" of the text in a high-dimensional space.

Stage 4: Vector Store & Retrieval. We use ChromaDB as our persistent vector database. When a user asks a question, their question is also turned into a vector. We then use a mathematical "Similarity Search" to find the 3 or 4 chunks in our database that are "closest" to the question's vector.

Stage 5: Augmented Generation. The retrieved chunks are placed into a "System Prompt" alongside the user's question. This "Augmented" prompt is sent to Llama 3.2, which then summarizes the chunks into a coherent answer.

7. Hardware and Software Requirements

To ensure the system runs smoothly in a production-like environment, the following requirements were strictly adhered to: **Hardware Requirements:**

- **CPU:** Intel Core i5 or AMD Ryzen 5 (10th Generation minimum) is required to handle the backend logic and PDF processing.
- **RAM:** 16GB DDR4 is highly recommended. While the system can run on 8GB, the OS and the LLM together often exceed this, leading to slow "swapping" on the disk.
- **Storage:** At least 20GB of SSD space. The LLM weights themselves take up about 5-8GB, and the vector database grows as more documents are added.
- **GPU (Optional but recommended):** An NVIDIA GPU with at least 6GB of VRAM (like an RTX 3060) allows the LLM to generate text 10 times faster than a CPU.

Software Requirements:

- **Backend:** Python 3.10+ is the core language. We use FastAPI for building high-performance asynchronous endpoints.
- **AI Engine:** Ollama is used as the model orchestrator. It manages the lifecycle of the Llama 3.2 model.
- **Vector Database:** ChromaDB (Open Source) for persistent storage of text embeddings.
- **Frontend:** A combination of HTML5, CSS3 (using Tailwind-like styling), and Vanilla JavaScript for a reactive, no-refresh user experience.
- **Libraries:** langchain for RAG orchestration, pypdf for file handling, and uvicorn as the web server.

8. System Architecture and Design

The system follows a Three-Tier Architecture pattern.

Tier 1: The Presentation Layer. This is the frontend dashboard. It features a "File Dropzone" for uploading PDFs and a "Chat Interface" for interacting with the AI. It uses the Backdrop-filter CSS property to give a modern, transparent look. Communication with the backend happens via asynchronous Fetch calls to ensure the page doesn't reload during processing.

Tier 2: The Logic Layer (API Server). Built with FastAPI, this layer manages the state of the application. It handles the routing of requests. When a file is uploaded, the Logic Layer triggers the "Ingestion Pipeline." When a query is received, it triggers the "Retrieval Pipeline." It acts as the "Brain" that coordinates between the user and the AI.

Tier 3: The Data & AI Layer. This is the foundation of the system. It consists of ChromaDB and the Ollama service. ChromaDB handles the "Long-term Memory" (the document vectors), while Ollama handles the "Reasoning" (the LLM). The design also includes a "Knowledge Routing" logic. If a user's query doesn't match any document chunks with a high confidence score, the system is designed to route the query to the LLM's general knowledge base or inform the user that no specific document context was found. This prevents the system from making up false information based on weak matches.

9. Project Timeline

The project was executed over a period of 12 weeks, following an Agile-like schedule:

- **Weeks 1-2: Literature Review.** Deep dive into the RAG research paper by Patrick Lewis and understanding the math behind Vector Embeddings. Selecting the right stack (FastAPI vs Flask, ChromaDB vs FAISS).
- **Weeks 3-4: Prototype Backend.** Setting up the Python environment and integrating Ollama. Successfully running a "Hello World" chat with Llama 3.2 in a terminal environment.
- **Weeks 5-6: Ingestion Engine.** Developing the PDF parsing and chunking logic. This involved testing different "chunk sizes" to see which one provided the best balance between context and speed.
- **Weeks 7-8: Vector Integration.** Connecting the chunking engine to ChromaDB. Implementing the first version of the "Retrieve-then-Generate" loop. Testing with 10-20 sample PDFs.
- **Weeks 9-10: Frontend & UI.** Designing the web dashboard. Implementing real-time feedback (like "Uploading..." and "AI is typing...") to improve the user experience.
- **Weeks 11-12: Testing & Documentation.** Performing hardware stress tests (comparing CPU vs GPU performance) and writing the final project report and synopsis. Finalizing the "One-Click" deployment script.

10. Implementation Results

The implementation yielded highly successful results across various benchmarks.

Ingestion Speed: The system was able to process a 100-page PDF (approx. 50,000 words) in about 45 seconds on a standard laptop. This included text extraction, chunking, and embedding generation.

Retrieval Accuracy: We tested the system with "Tricky" questions that required connecting information from Page 10 and Page 50. Thanks to the overlapping chunk methodology, the system successfully retrieved the correct context 92% of the time.

Inference Latency: On an 8GB RAM system (CPU only), the "Time to First Token" was approximately 4 seconds. On a GPU-enabled system, this dropped to under 0.5 seconds, providing a near-instant chat experience.

User Interface: The final web dashboard was tested with non-technical users who were able to upload documents and get answers without any instructions, proving the "Operational Feasibility" of our design. The "Dual-Mode" fallback worked as intended, providing general AI assistance when no documents were present in the library.

11. Conclusion and Future Scope

In conclusion, the "Enterprise AI Chatbot" project successfully demonstrates that high-quality, secure, and semantically-aware AI is no longer the exclusive domain of tech giants. By combining FastAPI, Ollama, and ChromaDB, we have built a robust tool that addresses the real-world problems of information overload and data privacy. This project has shown that RAG is a superior architecture for business use cases compared to simple fine-tuning, as it is cheaper, faster to update, and provides verifiable results.

Future Scope: The journey doesn't end here. The next logical step is **Multimodal RAG**. Future versions of this system will use models like Llama 3.2 Vision to not only read the text in PDFs but also "see" and interpret graphs, charts, and diagrams, which are common in corporate reports. Additionally, we plan to implement **Knowledge Graphs**. While vector search is great for similarity, a Knowledge Graph can understand relationships (e.g., "Project X belongs to Department Y"). Combining Vectors with Graphs will make the chatbot even more "human-like" in its reasoning. Finally, integrating **Active Learning**, where the chatbot learns from user corrections, will ensure the "Private Brain" gets smarter every single day it is used within the organization.

12. References

1. **Lewis, P., et al. (2020).** "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems (NeurIPS)*. The foundational paper for RAG.
2. **FastAPI Documentation (2024).** "High performance, easy to learn, fast to code, ready for production." <https://fastapi.tiangolo.com/>
3. **Ollama Documentation (2024).** "Run Llama 3.2, Mistral, and other large language models locally." <https://ollama.ai/>
4. **ChromaDB Documentation (2024).** "The AI-native open-source embedding database." <https://docs.trychroma.com/>
5. **Reimers, N., & Gurevych, I. (2019).** "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *EMNLP*. Essential for understanding the embedding process used in this project.
6. **Meta AI (2024).** "Llama 3.2: The next generation of open-source large language models." Technical Report.
7. **LangChain Python Documentation.** "Building applications with LLMs through composability." <https://python.langchain.com/>